Access-Induced Semantic Divergence in Generated Program Transformations

Anonymous Author(s)
Affiliation
Address
email

Abstract

Access-induced semantic divergence occurs when an operation that appears observational updates latent access state and changes a later externally visible result. We formalize this pattern as observation-sensitive deterministic systems (OSDS), prove bounded structural properties for a straight-line core, and evaluate it on real Python packages and generated program transformations. The real-code study confirms 20 divergences across 12 unmodified packages, with 9 caller-level branch effects and 19/19 mechanism controls eliminating divergence. A frozen primary coding-agent benchmark contains 13 base tasks and 26 normal/warned variants. Across three Codex task-model configurations, five model-task outputs were verified OSDS failures, all concentrated in two underlying package witnesses. The benchmark ordinary checks missed all five; OSDS-aware evaluation detected all five; mechanism-neutralizing controls reproduced and removed 5/5. A prospectively frozen expansion added 7 base tasks and 14 variants. Auditing the 42 prospective outputs separately, 23/42 were task-compliant transformations, 19/42 returned unchanged code, and 0 produced verified OSDS divergence. The results support a bounded conclusion: generated transformations can silently violate access-sensitive semantics on validated witnesses, and OSDS-aware replay plus causal controls provide a verifiable method for detecting and attributing this failure mode.

1 Introduction

Program transformations often treat logging, representation, inspection, caching, and repeated access as behavior-preserving. That treatment is valid when the accessed operation is observational in the semantic sense. It can fail when the operation updates latent state that a later read exposes. We call this pattern access-induced semantic divergence.

The phenomenon is related to observer effects and probe effects, but the paper studies a specific sequential mechanism: an observation-shaped access mutates latent access state, and a later program result changes. This matters for generated code because requested edits are often local and plausible. A model may add instrumentation, memoize a representation, or simplify repeated access while preserving ordinary tests that do not vary observation order.

We study this mechanism through three layers. The first layer is an observation-sensitive deterministic-system (OSDS) model. It separates exposed read values from latent access state and supports formal statements about determinism, zero-divergence conditions, and preservation of body-level divergence under a linear cap. The second layer is a real-code oracle over unmodified Python packages. It confirms executable package witnesses and caller-level effects. The third layer is a frozen generated-transformation study built from those witnesses.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

```
Original shape:
  with catching_logs(handler, level):
      result = subject(logger, handler)

Generated instrumentation shape:
  logger.debug("diagnostic")
  with catching_logs(handler, level):
      result = subject(logger, handler)
```

Figure 1: Compact pytest failure schema. The inserted log call looks like observation. In the verified Terra and Luna rows, it mutates logging/handler state that is read later. Ordinary benchmark checks passed; the OSDS oracle failed; isolating the diagnostic logger from the captured handler hierarchy removed the divergence for all four pytest rows.

The empirical claim is bounded. We do not estimate ecosystem prevalence, and the three tested Codex task-model configurations are not independent model providers. The main result is mechanism-grounded: in the primary benchmark, five generated outputs silently violated OSDS preservation under ordinary checks, and all five failures disappeared when the identified access-induced mechanism was neutralized while the generated code stayed fixed.

Contributions. We make four contributions. First, we give an OSDS formal model for access-state-carrying sequential systems. Second, we report 20 confirmed real-package divergences across 12 packages, including 9 caller-level branch effects. Third, we evaluate frozen generated transformations and identify five silent OSDS failures across Terra and Luna configurations, with zero false YES self-assessments in the primary study. Fourth, we correct the prospective expansion accounting by separating task compliance, ordinary correctness, and OSDS preservation: 23/42 prospective outputs were task-compliant transformations, 19/42 were unchanged, and 0/42 produced verified OSDS divergence.

2 Motivating Real-Package Example

Figure 1 shows the shortest verified failure family from the primary study. The task asked for behavior-preserving instrumentation around a pytest logging witness. The generated edit inserted diagnostic logging that looked observational. In the witness, that access interacted with the captured handler hierarchy used by `catching_logs`; the later handler-level behavior changed.

The example gives the intuition before the formal notation. The generated transformation did not crash, did not fail the ordinary smoke check, and did not look like a destructive update at the edit site. The divergence appeared only when the evaluation compared the later visible result across the access order that the witness exposes. In the causal-control replay, the exact generated files were reused. The only change was the witness environment: diagnostic logging was isolated from the handler hierarchy responsible for the latent mutation. Under that neutralized mechanism, the OSDS divergence disappeared.

This is a generated transformation failure, not a defect claim about pytest. It shows that a preservation checker for generated edits needs to model the latent state that observation-shaped accesses can affect.

3 Observation-Sensitive Deterministic Systems

An OSDS configuration contains a semantic value $x = (b, a, d)$ and an accumulator y. The component b is stable, a is an access count, and d is latent drift. A read exposes a value and updates access state:

$$\mathsf{read}(b, a, d, y) = ((b, a + 1, r(a, d)), y + f(b, a, d)).$$

An observation exposes no additive value but can update latent drift:

$$\mathsf{obs}(b, a, d, y) = ((b, a, g(d)), y).$$

A program body is a deterministic fold over a finite operation sequence $\pi \in \{\mathsf{read}, \mathsf{obs}\}^*$. A cap C maps the final body configuration to the externally visible result.

Stage	Baseline path	Transformed path	OSDS question
Initialize	matched object	matched object	same visible setup
Observe	no extra access	observation-shaped access	can latent state drift?
Read	later visible read	later visible read	does exposed value change?
Classify	ordinary check	OSDS comparison	is divergence access-induced?

Figure 2: OSDS evaluation compares access-order variants that ordinary single-order checks may not exercise.

For a fixed initial configuration cfg_0, body sequence π, and cap C, let $[\![\pi]\!]_C(\mathsf{cfg}_0)$ denote the final visible output. Given a multiset of operations M and an admissible ordering set $\Pi(M)$, semantic divergence is

$$\Delta_C(M, \mathsf{cfg}_0) = \max_{\pi \in \Pi(M)} [\![\pi]\!]_C(\mathsf{cfg}_0) - \min_{\pi \in \Pi(M)} [\![\pi]\!]_C(\mathsf{cfg}_0).$$

A transformation is OSDS-preserving for a witness when the baseline and candidate agree under the witness's OSDS relation. In the benchmark, this relation is implemented as a package-specific metamorphic oracle and then manually reviewed for access-induced attribution.

We use two evidence roles. Hidden-observation rows contain accesses that look observational in the requested edit and reveal their effect only through later state. Expected-access-sensitive rows contain operations whose consumption or mutation should be visible from the code shape. The primary verified OSDS failures all occurred in hidden-observation rows.

4 Structural Results

The formal core is a straight-line deterministic template. Within that scope, fixed-order execution is deterministic: induction over the operation sequence gives a unique next configuration at each step. Two zero-divergence conditions follow directly. If $g(d) = d$, observations are inert and can be moved without changing the reduced read sequence. If $f(b, a, d) = h(b)$, reads ignore access count and drift, so every ordering with the same number of reads produces the same body accumulator.

A linear cap preserves body-level divergence. For $C(y) = \alpha y + \beta$ with $\alpha \neq 0$, $y_1 \neq y_2$ implies $C(y_1) - C(y_2) = \alpha(y_1 - y_2) \neq 0$. This proposition is the only cap theorem used as an unbounded formal claim. Nonlinear amplification and broad degree laws are treated as bounded computational findings.

The appendix states a restricted adjacent-swap result for the monotone positive-parameter fragment. Under deterministic updates, monotone positive observation drift, and monotone access-sensitive reads, extrema over the checked order set occur at boundary orderings obtained by adjacent swaps that move observations before or after reads. The appendix also states the restricted range-degree result supported by exact rational checks: for checked compositional OSDS cases $m = 1..4$, cap degree $d = 1..5$, and $L = 2..15$, extremal-order interpolation detects degree $2d$. This is not a theorem for all OSDS programs.

5 Real-Code Study

The real-code study uses static screening only as a review queue. The screen covered 73 PyPI packages and 278 reviewed MEDIUM/HIGH findings. Manual review labeled 203 likely true positives and 75 likely false positives, for reviewed precision 0.7302. A rebuilt exact-version source snapshot contained 4,383 analyzable classes, and a deterministic sample of 200 unflagged classes found 0 likely missed matches. These numbers are audit evidence rather than prevalence evidence.

Executable confirmation is the decisive filter. The oracle selected 60 candidates, constructed 39 executable package-shaped harnesses, and confirmed 20 divergences across 12 unmodified packages: httpcore, PyYAML, pytest, markdown, more-itertools, docutils, beautifulsoup4, boltons, cerberus, dnspython, h11, and anyio. The divergences include stream materialization, identity-cache reuse, handler-level mutation, reference-registry mutation, cursor advance, destructive tree extraction, cache recency/statistics, validation-error population, and buffer consumption.

Real-code evidence item	Count
Reviewed static findings	278
Executable harnesses	39
Confirmed divergences	20
Confirmed packages	12
Caller-level wrappers changing downstream execution	9
Mechanism controls eliminating divergence	19/19

Table 1: Real-code OSDS evidence. Static screening is a review queue; executable harnesses and controls provide the semantic evidence.

Model	Exec.	Pres.	Ord. bugs	Invalid	Ver. OSDS	Silent
gpt-5.6-sol	26	23	3	0	0	0
gpt-5.6-terra	24	18	4	2	2	2
gpt-5.6-luna	24	17	4	2	3	3

Table 2: Primary frozen coding-agent results. All verified OSDS failures passed the benchmark ordinary checks and failed OSDS-aware evaluation.

Nine confirmed divergences were lifted into caller-level wrappers that changed branch labels and downstream consequences. Nineteen controls eliminated divergence under fresh-object, reset-between, or pure-observation interventions. These controls show that the detected differences depend on the access-induced mechanism named by the witness.

6 Generated Transformation Study

The primary benchmark was frozen before model execution. It contains 13 base tasks and 26 normal/warned variants from 9 packages and 9 unique witnesses. The six transformation families are instrumentation, caching/materialization, refactoring, repeated-access cleanup, access reordering, and debugging/inspection. Each generation task was fresh and projectless. The model saw only the frozen prompt, with no OSDS labels, oracle output, witness labels, benchmark explanation, or previous responses.

Replay treats generated text as data. Raw responses are saved exactly, code is extracted, ordinary smoke checks are run, and then OSDS-aware checks compare baseline and candidate behavior under the witness relation. Manual review classifies non-preserved rows as verified semantic divergence, ordinary programming bug, invalid patch, environment failure, oracle issue, or unclear.

The five verified OSDS failures are five model-task outcomes over two underlying package witnesses. Terra failed on two pytest instrumentation rows. Luna failed on those two pytest rows and one PyYAML caching/materialization row. Sol preserved the verified-OSDS rows on which Terra or Luna diverged. Expected-access-sensitive rows produced ordinary bugs or invalid patches, with no verified OSDS divergence.

Self-assessment was secondary. Across Sol, Terra, and Luna, there were zero false YES preservation claims on verified OSDS divergences. Conservative NO responses were common, so self-assessment should not replace execution replay.

7 Causal Attribution and Prospective Expansion

The causal-control experiment replays the exact generated files for the five verified failures. The candidate source remains unchanged. The intervention changes only the witness mechanism that caused access-induced divergence.

This supports the access-induced attribution. The failures are mechanism-dependent generated-code defects: ordinary checks pass, OSDS checks fail, and targeted neutralization removes divergence while preserving the generated candidate.

Family	Intervention	Rows	Result
pytest instrumentation	isolate diagnostic logger from captured handler hierarchy	4	4/4 removed
PyYAML caching	clear representer identity cache after observation	1	1/1 removed

Table 3: Causal controls. Under the original witness all five failures reproduced; under mechanism-neutralizing controls all five OSDS divergences disappeared.

Model	Outputs	Task-compliant	Unchanged	Ordinary pass	Ver. OSDS
gpt-5.6-sol	14	8	6	14	0
gpt-5.6-terra	14	8	6	14	0
gpt-5.6-luna	14	7	7	14	0
Total	42	23	19	42	0

Table 4: Prospective expansion with task compliance separated from semantic preservation. Unchanged outputs are OSDS-preserving but are not counted as task-compliant transformations.

After the causal phase, 11 unused confirmed real-code witnesses were audited prospectively. Seven met eligibility criteria and were frozen before model execution, producing 14 normal/warned variants from boltons, dnspython, and h11. Sol, Terra, and Luna were run on these exact prompts as fresh projectless tasks. Self-assessment was skipped for the expansion.

The prospective benchmark produced no new verified OSDS divergences. The corrected interpretation is narrower than a 42/42 transformation success claim: 23/42 outputs performed a nontrivial task-compliant transformation, 19/42 returned unchanged code, and 23/23 task-compliant outputs were OSDS-preserving. The result constrains failure incidence among these frozen responses and supports reporting the primary failures as reproducible examples rather than as a prevalence estimate.

8 Related Work

Observer effects and Heisenbugs. Heisenbugs and probe effects describe programs whose behavior changes under observation or debugging. Musuvathi et al. discuss debugging statements affecting concurrent interleavings in systematic testing [Musuvathi et al., 2008]. Sallinger, Weissenbacher, and Zuleger formalize Heisenbugs as hyperproperties and analyze causes including probe effects and nondeterminism [Sallinger et al., 2026]. OSDS is narrower: it models sequential access-state mechanisms where observation-shaped operations mutate latent state and later reads expose the difference.

Semantic preservation and equivalence. Compiler verification and translation validation study semantic preservation between source and transformed programs [Pnueli et al., 1998, Leroy, 2009]. OSDS uses the same preservation motivation but focuses on generated source-level transformations over library objects whose read and observation operations carry latent state. The oracle compares access-order behavior rather than proving full program equivalence.

Metamorphic testing. Metamorphic testing addresses oracle problems by checking relations across related executions [Chen et al., 1998, 2018]. The OSDS-aware oracle is a specialized metamorphic relation: the follow-up execution changes the placement of an observation-shaped access while keeping witness setup matched. Generic metamorphic testing supplies the testing pattern; OSDS supplies the mechanism and classification boundary.

Generated code and repair verification. HumanEval and Codex made execution-based evaluation central for code generation [Chen et al., 2021]. SWE-bench extends evaluation to issue-resolution patches in real repositories [Jimenez et al., 2024]. Automated program repair has long shown that patches can overfit tests [Le Goues et al., 2012, Smith et al., 2015]. This paper studies a different preservation failure: a generated transformation can pass the ordinary benchmark tests while violating an access-state relation that those tests do not exercise.

9 Limitations

The coding-agent benchmark is small and correlated by witness and package. Counts are benchmark outcomes, not prevalence estimates. The five verified primary failures are five model-task failures over two underlying package witnesses. The prospective expansion added seven witnesses and found zero new verified OSDS divergences.

The three model labels are Codex task-model configurations reported by the Codex runtime. They were collected through fresh projectless Codex tasks rather than the OpenAI API, and temperature/seed were not exposed. The study is Python-focused. Manual review remains part of verified OSDS classification. The formal results apply to the stated straight-line deterministic core; broader analyzer and range-degree claims remain empirical or bounded computational evidence.

10 Conclusion

Access-induced semantic divergence is a concrete preservation risk for generated transformations on access-state-carrying software objects. The OSDS model identifies the mechanism, the real-code study confirms package witnesses, and the coding-agent benchmark shows five silent generated failures under ordinary checks. Exact replay plus mechanism-neutralizing controls reproduced and removed all five primary failures. The prospective expansion found no new OSDS divergences and exposed substantial unchanged-output behavior. The resulting assessment is bounded: OSDS-aware verification provides a reproducible method for detecting and attributing a semantic failure mode that ordinary checks missed in this benchmark, without supporting broad claims of model unsafety.

References

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021.

T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology, 1998.

T. Y. Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018. doi: 10.1145/3143561.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=VTF8yNQM66`.

Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.

Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. doi: 10.1145/1538788.1538814.

Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtiu. Finding and reproducing heisenbugs in concurrent programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation*, pages 267–280, 2008.

Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 1384 of *Lecture Notes in Computer Science*, pages 151–166. Springer, 1998. doi: 10.1007/BFb0054170.

Sarah Sallinger, Georg Weissenbacher, and Florian Zuleger. A formalization of heisenbugs and their causes in terms of hyperproperties. *Software and Systems Modeling*, 2026. doi: 10.1007/s10270-025-01345-7.

Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 532–543, 2015. doi: 10.1145/2786805.2786825.

A Formal Details

A.1 State and transitions

An OSDS state is $s = (b, a, d, y)$ with base b, access count a, latent drift d, and accumulator y. A read transition is deterministic:

$$R(s) = (b, a + 1, r(a, d), y + f(b, a, d)).$$

An observation transition is deterministic:

$$O(s) = (b, a, g(d), y).$$

For a finite sequence $\pi = o_1 \dots o_n$, composition is $[\![\epsilon]\!](s) = s$ and $[\![\pi o]\!](s) = o([\![\pi]\!](s))$. Branch evaluation is a deterministic predicate $B(C([\![\pi]\!](s)))$ over the capped visible result or final configuration. Branch divergence occurs when two admissible orderings produce different branch labels or downstream consequences.

A.2 Semantic divergence

Given operation multiset M, admissible orderings $\Pi(M)$, initial state s_0, and deterministic cap C, define $V_C(\pi, s_0) = C([\![\pi]\!](s_0))$. The access-order range is $\Delta_C(M, s_0) = \max_{\pi \in \Pi(M)} V_C(\pi, s_0) - \min_{\pi \in \Pi(M)} V_C(\pi, s_0)$ for finite $\Pi(M)$. A transformation preserves the witness when baseline and candidate have equal OSDS-observable results under the witness relation.

A.3 Fixed-order determinism

Proposition. For fixed s_0, fixed π, deterministic R, deterministic O, and deterministic C, $V_C(\pi, s_0)$ is unique. **Proof.** Induct on $|\pi|$. The empty sequence returns s_0. For πo, the induction hypothesis gives a unique intermediate state and $o \in \{R, O\}$ maps it to a unique next state. The deterministic cap gives a unique output.

A.4 Zero divergence under no observation-induced mutation

Proposition. If $g(d) = d$ and observations expose no additive value, moving observations within a fixed multiset does not change the body accumulator or final latent state. **Proof sketch.** Each observation is the identity on (b, a, d, y). Removing observations leaves the same sequence of read operations. The same deterministic read sequence is executed from the same state, so the accumulator and final state match.

A.5 Zero divergence under access-insensitive reads

Proposition. If $f(b, a, d) = h(b)$, every ordering with L reads has the same body accumulator $Lh(b)$. **Proof.** Every read contributes the same exposed value for fixed b regardless of access count or drift. Observations may change d, but d is ignored by f. A deterministic cap depending only on the accumulator and final counts preserves equality.

A.6 Linear-cap preservation

Proposition. Let $C(y) = \alpha y + \beta$ with $\alpha \neq 0$. If body accumulators $y_1 \neq y_2$, then $C(y_1) \neq C(y_2)$. **Proof.** $C(y_1) - C(y_2) = \alpha(y_1 - y_2)$, which is nonzero under exact arithmetic.

A.7 Adjacent-swap extrema under monotone access drift

Restricted result. Consider the positive-parameter fragment with identical reads, identical observations, deterministic updates, monotone observation drift, and reads monotone in drift. If a neighboring pair is RO, swapping it to OR cannot decrease the later drift seen by subsequent reads. Repeated adjacent swaps move observations toward the front to reach a maximum ordering, or toward the back to reach a minimum ordering. **Proof sketch.** The swap changes only the relative placement of one observation and one read. Under monotone positive drift, the read in OR sees

the observation-induced drift that it misses in RO; later reads see a state no smaller in the order. Repeating the local monotone step yields boundary extrema. **Scope.** This result assumes monotone positive parameters and identical operations. It is not used for arbitrary Python orderings.

A.8 Restricted 2d range-degree statement

Checked theorem. For the compositional OSDS cap family implemented in the exact-rational validation scripts, with $m = 1..4$, cap degree $d = 1..5$, and $L = 2..15$, exhaustive checks on small feasible grids plus extremal-order interpolation detect access-order range degree $2d$. **Derivation.** In the checked family, the body-range term is $\eta m L(L-1)/2$. The compositional cap contributes $d-1$ final read factors, each quadratic in L. The leading degree is therefore $2 + 2(d-1) = 2d$. Exact rational scripts verify the extremal formula against enumerated feasible orderings and holdout values on the stated grid. **Scope.** This is a restricted checked result for the named family and grid. It replaces the earlier unqualified degree claim.

B Benchmark Details

Primary tasks are in `benchmark/tasks.jsonl` under the agent-behavior study tree. Primary prompt files are in that tree's `prompts/` directory. The pre-model manifest and raw Codex task-model responses are in `external_collection/` and `external_collection/responses/`.

Manual review rows are in the primary study analysis directory as the 20260813 manual-review JSONL. The matching cross-model summary is stored beside it as JSON and Markdown. Prospective tasks, prompts, and raw responses are under `benchmark_expansion/`. Exact prospective replay outputs are in the three `*expansion-exact-20260813Tcutscope` result directories.

The prospective task-compliance audit is `analysis/prospective_task_compliance.csv`. It separates `task_compliance`, `ordinary_correctness`, and `osds_preservation` for all 42 prospective outputs. The supplement artifact index maps these shortened descriptions to exact relative paths.

C Reproducibility

The reviewer supplement contains a relative-path artifact index, frozen prompts, raw response JSONLs, replay summaries, causal-control scripts, real-code oracle metadata, and expected outputs. It excludes virtual environments, caches, authentication material, and personal absolute paths. Replays require Python 3.11 or newer with pytest and the package versions named by the witness manifests. The supplement `REPRODUCE.md` gives exact commands for the primary replay, prospective replay, causal-control replay, and table reproduction.